

Complete Scikit-Learn Cheat Sheet

Table of Contents

1. Installation & Imports
 2. Data Loading & Preprocessing
 3. Train-Test Split
 4. Feature Scaling
 5. Encoding Categorical Variables
 6. Feature Engineering
 7. Supervised Learning - Classification
 8. Supervised Learning - Regression
 9. Unsupervised Learning
 10. Model Evaluation Metrics
 11. Cross-Validation
 12. Hyperparameter Tuning
 13. Pipelines
 14. Model Persistence
-

1. Installation & Imports

```
python

# Installation
pip install scikit-learn

# Core imports
import numpy as np
import pandas as pd
from sklearn import datasets
from sklearn.model_selection import train_test_split, cross_val_score, GridSearchCV
from sklearn.preprocessing import StandardScaler, MinMaxScaler, LabelEncoder, OneHotEncoder
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

2. Data Loading & Preprocessing

Loading Built-in Datasets

```
python
```

```
# Load sample datasets
```

```
from sklearn.datasets import load_iris, load_boston, load_digits, make_classification
```

```
# Classification dataset
```

```
iris = load_iris()
```

```
X, y = iris.data, iris.target
```

```
# Regression dataset
```

```
california = fetch_california_housing()
```

```
X, y = california.data, california.target
```

```
# Generate synthetic data
```

```
X, y = make_classification(n_samples=1000, n_features=20, n_classes=2, random_state=42)
```

Handling Missing Values

```
python
```

```
from sklearn.impute import SimpleImputer
```

```
# Impute with mean
```

```
imputer = SimpleImputer(strategy='mean') # Options: 'mean', 'median', 'most_frequent', 'constant'
```

```
X_imputed = imputer.fit_transform(X)
```

```
# Impute with specific value
```

```
imputer = SimpleImputer(strategy='constant', fill_value=0)
```

```
X_imputed = imputer.fit_transform(X)
```

3. Train-Test Split

```
python
```

```
from sklearn.model_selection import train_test_split

# Basic split (80-20)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Stratified split (maintains class distribution)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, stratify=y, random_state=42)

# Train-Validation-Test split
X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.3, random_state=42)
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.5, random_state=42)
```

4. Feature Scaling

```
python

from sklearn.preprocessing import StandardScaler, MinMaxScaler, RobustScaler

# StandardScaler (mean=0, std=1)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test) # Use same scaler, don't fit again!

# MinMaxScaler (scale to range [0,1])
scaler = MinMaxScaler()
X_scaled = scaler.fit_transform(X_train)

# MinMaxScaler with custom range
scaler = MinMaxScaler(feature_range=(0, 10))
X_scaled = scaler.fit_transform(X_train)

# RobustScaler (robust to outliers)
scaler = RobustScaler()
X_scaled = scaler.fit_transform(X_train)
```

5. Encoding Categorical Variables

```
python
```

```
from sklearn.preprocessing import LabelEncoder, OneHotEncoder
```

```
# LabelEncoder (for target variable or ordinal features)
```

```
le = LabelEncoder()
```

```
y_encoded = le.fit_transform(y) # ['cat', 'dog', 'cat'] -> [0, 1, 0]
```

```
y_original = le.inverse_transform(y_encoded) # Decode back
```

```
# OneHotEncoder (for nominal features)
```

```
ohe = OneHotEncoder(sparse_output=False)
```

```
X_encoded = ohe.fit_transform(X_categorical)
```

```
# Get feature names
```

```
feature_names = ohe.get_feature_names_out(['color'])
```

```
# Using pandas get_dummies (alternative)
```

```
df_encoded = pd.get_dummies(df, columns=['color', 'size'])
```

6. Feature Engineering

Feature Selection

```
python
```

```
from sklearn.feature_selection import SelectKBest, chi2, f_classif, RFE
```

```
from sklearn.ensemble import RandomForestClassifier
```

```
# SelectKBest - Select top k features
```

```
selector = SelectKBest(f_classif, k=10)
```

```
X_new = selector.fit_transform(X, y)
```

```
selected_features = selector.get_support(indices=True)
```

```
# Recursive Feature Elimination (RFE)
```

```
estimator = RandomForestClassifier()
```

```
rfe = RFE(estimator, n_features_to_select=10)
```

```
X_new = rfe.fit_transform(X, y)
```

Polynomial Features

```
python
```

```
from sklearn.preprocessing import PolynomialFeatures

# Create polynomial features
poly = PolynomialFeatures(degree=2, include_bias=False)
X_poly = poly.fit_transform(X)
```

7. Supervised Learning - Classification

Logistic Regression

```
python

from sklearn.linear_model import LogisticRegression

# Create and train
model = LogisticRegression(max_iter=1000, random_state=42)
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)
y_pred_proba = model.predict_proba(X_test) # Probability estimates

# Evaluate
accuracy = model.score(X_test, y_test)
```

Decision Tree Classifier

```
python

from sklearn.tree import DecisionTreeClassifier

model = DecisionTreeClassifier(max_depth=5, min_samples_split=10, random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

# Feature importance
importances = model.feature_importances_
```

Random Forest Classifier

```
python
```

```
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(n_estimators=100, max_depth=10, random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

# Feature importance
importances = model.feature_importances_
```

Support Vector Machine (SVM)

```
python

from sklearn.svm import SVC

# Linear kernel
model = SVC(kernel='linear', C=1.0, random_state=42)

# RBF kernel
model = SVC(kernel='rbf', C=1.0, gamma='scale', random_state=42)

model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

K-Nearest Neighbors (KNN)

```
python

from sklearn.neighbors import KNeighborsClassifier

model = KNeighborsClassifier(n_neighbors=5, weights='uniform') # weights: 'uniform', 'distance'
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

Naive Bayes

```
python
```

```
from sklearn.naive_bayes import GaussianNB, MultinomialNB
```

```
# Gaussian Naive Bayes
```

```
model = GaussianNB()
```

```
model.fit(X_train, y_train)
```

```
# Multinomial Naive Bayes (for count data)
```

```
model = MultinomialNB()
```

```
model.fit(X_train, y_train)
```

Gradient Boosting

```
python
```

```
from sklearn.ensemble import GradientBoostingClassifier
```

```
model = GradientBoostingClassifier(n_estimators=100, learning_rate=0.1, max_depth=3, random_state=42)
```

```
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
```

8. Supervised Learning - Regression

Linear Regression

```
python
```

```
from sklearn.linear_model import LinearRegression
```

```
model = LinearRegression()
```

```
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
```

```
# Coefficients and intercept
```

```
coefficients = model.coef_
```

```
intercept = model.intercept_
```

Ridge Regression (L2 Regularization)

```
python
```

```
from sklearn.linear_model import Ridge

model = Ridge(alpha=1.0) # Higher alpha = more regularization
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

Lasso Regression (L1 Regularization)

```
python

from sklearn.linear_model import Lasso

model = Lasso(alpha=0.1)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

ElasticNet (L1 + L2)

```
python

from sklearn.linear_model import ElasticNet

model = ElasticNet(alpha=0.1, l1_ratio=0.5) # l1_ratio: mix of L1 and L2
model.fit(X_train, y_train)
```

Decision Tree Regressor

```
python

from sklearn.tree import DecisionTreeRegressor

model = DecisionTreeRegressor(max_depth=5, random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

Random Forest Regressor

```
python

from sklearn.ensemble import RandomForestRegressor

model = RandomForestRegressor(n_estimators=100, max_depth=10, random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

Support Vector Regression (SVR)

```
python

from sklearn.svm import SVR

model = SVR(kernel='rbf', C=1.0, gamma='scale')
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

Gradient Boosting Regressor

```
python

from sklearn.ensemble import GradientBoostingRegressor

model = GradientBoostingRegressor(n_estimators=100, learning_rate=0.1, max_depth=3, random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

9. Unsupervised Learning

K-Means Clustering

```
python

from sklearn.cluster import KMeans

model = KMeans(n_clusters=3, random_state=42)
clusters = model.fit_predict(X)

# Cluster centers
centers = model.cluster_centers_

# Inertia (sum of squared distances to nearest cluster center)
inertia = model.inertia_
```

DBSCAN

```
python
```

```
from sklearn.cluster import DBSCAN
```

```
model = DBSCAN(eps=0.5, min_samples=5)
```

```
clusters = model.fit_predict(X)
```

Hierarchical Clustering

```
python
```

```
from sklearn.cluster import AgglomerativeClustering
```

```
model = AgglomerativeClustering(n_clusters=3, linkage='ward') # linkage: 'ward', 'complete', 'average'
```

```
clusters = model.fit_predict(X)
```

Principal Component Analysis (PCA)

```
python
```

```
from sklearn.decomposition import PCA
```

```
# Reduce to 2 components
```

```
pca = PCA(n_components=2)
```

```
X_pca = pca.fit_transform(X)
```

```
# Variance explained
```

```
explained_variance = pca.explained_variance_ratio_
```

```
# Keep components that explain 95% variance
```

```
pca = PCA(n_components=0.95)
```

```
X_pca = pca.fit_transform(X)
```

t-SNE

```
python
```

```
from sklearn.manifold import TSNE
```

```
tsne = TSNE(n_components=2, random_state=42)
```

```
X_tsne = tsne.fit_transform(X)
```

10. Model Evaluation Metrics

Classification Metrics

```
python

from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, classification_report, roc_auc_score, roc_curve
)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)

# Precision, Recall, F1-Score
precision = precision_score(y_test, y_pred, average='weighted')
recall = recall_score(y_test, y_pred, average='weighted')
f1 = f1_score(y_test, y_pred, average='weighted')

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

# Classification Report (all metrics)
report = classification_report(y_test, y_pred)
print(report)

# ROC-AUC Score (binary classification)
roc_auc = roc_auc_score(y_test, y_pred_proba[:, 1])

# ROC Curve
fpr, tpr, thresholds = roc_curve(y_test, y_pred_proba[:, 1])
```

Regression Metrics

```
python
```

```
from sklearn.metrics import (
    mean_squared_error, mean_absolute_error, r2_score,
    mean_absolute_percentage_error
)

# Mean Squared Error
mse = mean_squared_error(y_test, y_pred)

# Root Mean Squared Error
rmse = mean_squared_error(y_test, y_pred, squared=False)

# Mean Absolute Error
mae = mean_absolute_error(y_test, y_pred)

# R-squared Score
r2 = r2_score(y_test, y_pred)

# Mean Absolute Percentage Error
mape = mean_absolute_percentage_error(y_test, y_pred)
```

11. Cross-Validation

```
python
```

```
from sklearn.model_selection import cross_val_score, cross_validate, KFold, StratifiedKFold

# Simple cross-validation (default 5-fold)
scores = cross_val_score(model, X, y, cv=5)
print(f'Accuracy: {scores.mean():.3f} (+/- {scores.std():.3f})')

# Cross-validation with specific metric
scores = cross_val_score(model, X, y, cv=5, scoring='f1_weighted')

# Multiple metrics
scoring = ['accuracy', 'precision_weighted', 'recall_weighted']
scores = cross_validate(model, X, y, cv=5, scoring=scoring)

# K-Fold
kfold = KFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(model, X, y, cv=kfold)

# Stratified K-Fold (maintains class distribution)
skfold = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(model, X, y, cv=skfold)

# Leave-One-Out Cross-Validation
from sklearn.model_selection import LeaveOneOut
loo = LeaveOneOut()
scores = cross_val_score(model, X, y, cv=loo)
```

12. Hyperparameter Tuning

Grid Search

```
python
```

```
from sklearn.model_selection import GridSearchCV

# Define parameter grid
param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [5, 10, 15, None],
    'min_samples_split': [2, 5, 10]
}

# Create GridSearchCV
grid_search = GridSearchCV(
    RandomForestClassifier(random_state=42),
    param_grid,
    cv=5,
    scoring='accuracy',
    n_jobs=-1, # Use all processors
    verbose=1
)

# Fit
grid_search.fit(X_train, y_train)

# Best parameters and score
print(f'Best parameters: {grid_search.best_params_}')
print(f'Best score: {grid_search.best_score_:.3f}')

# Best model
best_model = grid_search.best_estimator_

# Predict with best model
y_pred = best_model.predict(X_test)
```

Randomized Search

```
python
```

```
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import randint, uniform

# Define parameter distributions
param_distributions = {
    'n_estimators': randint(50, 500),
    'max_depth': randint(5, 50),
    'min_samples_split': randint(2, 20),
    'min_samples_leaf': randint(1, 10),
    'max_features': uniform(0.1, 0.9)
}

# Create RandomizedSearchCV
random_search = RandomizedSearchCV(
    RandomForestClassifier(random_state=42),
    param_distributions,
    n_iter=100, # Number of parameter settings sampled
    cv=5,
    scoring='accuracy',
    n_jobs=-1,
    random_state=42,
    verbose=1
)

# Fit
random_search.fit(X_train, y_train)

# Best parameters
print(f"Best parameters: {random_search.best_params_}")
best_model = random_search.best_estimator_
```

13. Pipelines

Basic Pipeline

```
python
```

```
from sklearn.pipeline import Pipeline

# Create pipeline
pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('classifier', RandomForestClassifier(random_state=42))
])

# Fit and predict
pipeline.fit(X_train, y_train)
y_pred = pipeline.predict(X_test)
accuracy = pipeline.score(X_test, y_test)
```

Pipeline with Preprocessing

```
python
```

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

# Define column types
numeric_features = ['age', 'income']
categorical_features = ['gender', 'city']

# Create transformers
numeric_transformer = Pipeline([
    ('scaler', StandardScaler())
])

categorical_transformer = Pipeline([
    ('onehot', OneHotEncoder(handle_unknown='ignore'))
])

# Combine transformers
preprocessor = ColumnTransformer([
    ('num', numeric_transformer, numeric_features),
    ('cat', categorical_transformer, categorical_features)
])

# Full pipeline
pipeline = Pipeline([
    ('preprocessor', preprocessor),
    ('classifier', LogisticRegression())
])

pipeline.fit(X_train, y_train)
```

Pipeline with GridSearch

```
python
```

```
# Create pipeline
pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('classifier', SVC())
])

# Define parameters (use double underscore for pipeline steps)
param_grid = {
    'classifier__C': [0.1, 1, 10],
    'classifier__kernel': ['linear', 'rbf'],
    'classifier__gamma': ['scale', 'auto']
}

# GridSearch with pipeline
grid_search = GridSearchCV(pipeline, param_grid, cv=5)
grid_search.fit(X_train, y_train)

best_model = grid_search.best_estimator_
```

14. Model Persistence

Save and Load Models

```
python

import pickle
import joblib

# Using joblib (recommended for scikit-learn)
joblib.dump(model, 'model.pkl')
loaded_model = joblib.load('model.pkl')

# Using pickle
with open('model.pkl', 'wb') as f:
    pickle.dump(model, f)

with open('model.pkl', 'rb') as f:
    loaded_model = pickle.load(f)

# Verify loaded model
y_pred = loaded_model.predict(X_test)
```

Common Workflow Example

python

1. Import libraries

```
import pandas as pd
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix
import joblib
```

2. Load data

```
df = pd.read_csv('data.csv')
X = df.drop('target', axis=1)
y = df['target']
```

3. Split data

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
```

4. Scale features

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

5. Create and train model

```
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train_scaled, y_train)
```

6. Make predictions

```
y_pred = model.predict(X_test_scaled)
```

7. Evaluate

```
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

8. Hyperparameter tuning

```
param_grid = {'n_estimators': [50, 100, 200], 'max_depth': [5, 10, None]}
grid_search = GridSearchCV(model, param_grid, cv=5)
grid_search.fit(X_train_scaled, y_train)
```

9. Save model

```
joblib.dump(grid_search.best_estimator_, 'best_model.pkl')
```

10. Load and use

```
loaded_model = joblib.load('best_model.pkl')
new_predictions = loaded_model.predict(new_data_scaled)
```

Quick Reference: Algorithm Selection

Classification:

- Small dataset + need interpretability → Logistic Regression, Decision Tree
- Large dataset + high accuracy → Random Forest, Gradient Boosting
- Text classification → Naive Bayes
- Non-linear boundaries → SVM with RBF kernel

Regression:

- Linear relationships → Linear Regression
- Prevent overfitting → Ridge/Lasso
- Non-linear relationships → Random Forest Regressor, Gradient Boosting

Clustering:

- Know number of clusters → K-Means
- Don't know number + arbitrary shapes → DBSCAN
- Hierarchical structure → Agglomerative Clustering

Dimensionality Reduction:

- Linear reduction → PCA
 - Non-linear visualization → t-SNE
-

Tips and Best Practices

1. **Always scale features** for distance-based algorithms (KNN, SVM, Linear models)
2. **Use stratified split** for imbalanced datasets
3. **Never fit scalers on test data** - only transform
4. **Use cross-validation** to get robust performance estimates
5. **Start simple** then increase complexity
6. **Use pipelines** to prevent data leakage
7. **Set random_state** for reproducibility
8. **Check for overfitting** using train vs validation performance
9. **Handle class imbalance** with stratification, SMOTE, or class weights
10. **Save preprocessing objects** along with models

